

LECTURE 5 · WEB TECHNOLOGIES

CSS

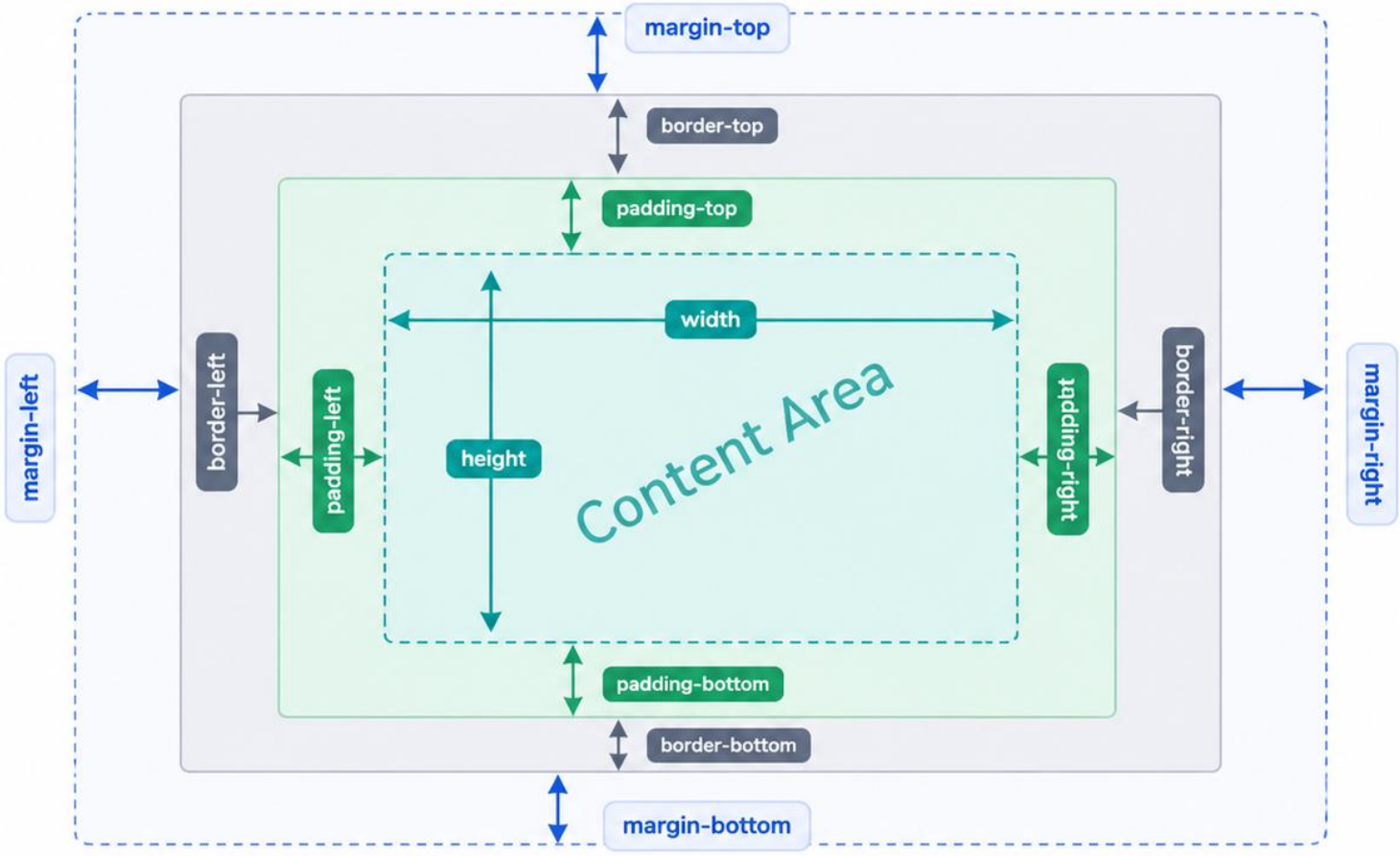
Md Istakiak Adnan Palash
Lecturer, Dept. of ICT
Bangladesh University of Professionals (BUP)

CSS Box Model

- CSS box model typically describes how these rectangular boxes are laid out on a web page.
- These boxes can have different properties and can interact with each other in different ways, but every box has a content area and optional surrounding padding, border, and margin areas.

CSS Box Model

- Every CSS box is divided into regions, consisting of:
  - **Margin:** Clears an area around the border. The margin does not have a background color, and it is completely transparent
  - **Border:** A border that lies around the padding and content. The border is affected by the background color of the box
  - **Padding:** Clears an area around the content. The padding is affected by the background color of the box
  - **Content:** The content of the box, where text and images appear



Total Width = margin-left + border-left + padding-left + width + padding-right + border-right + margin-right

Total Height = margin-top + border-top + padding-top + height + padding-bottom + border-bottom + margin-bottom

Setting the Width and Height

- The width and height property defines the width and height of the content area of an element.

- Example:

```
div {
   width: 300px;
   height: 200px;
}
```

CSS Outline

- An outline is a line that is drawn around elements, OUTSIDE the borders, to make the element "stand out".
- CSS has the following outline properties:
  - **outline-style:** Specifies the style of the outline
  - **outline-color:** Specifies the color of the outline
  - **outline-width:** Specifies the width of the outline
  - **outline-offset:** Adds space between the outline and the edge/border of an element
  - **outline:** A shorthand property

Outlines Vs Borders

- An outline looks very similar to the border, but it differs from border in the following ways:
  - Outlines do not take up space, because they always placed on top of the box of the element which may cause them to overlap other elements on the page.
  - Unlike borders, outlines won't allow us to set each edge to a different width, or set different colors and styles for each edge. An outline is the same on all sides.
  - Outlines do not have any impact on surrounding elements apart from overlapping.
  - Unlike borders, outlines do not change the size or position of the element.
  - Outlines may be non-rectangular, but you cannot create circular outlines.

CSS Overflow

- There may be a situation when the content of an element might be larger than the dimensions of the box itself.
- For example, given width and height properties did not allow enough room to accommodate the content of the element.
- CSS overflow property allowing you to specify whether to clip content, render scroll bars or display overflow content of a block-level element.
- This property can take one of the following values: visible (default), hidden, scroll, and auto.

CSS Overflow

Value	Description
visible	The default value. Content is not clipped; it will be rendered outside the element's box, and may thus overlap other content.
hidden	Content that overflows the element's box is clipped and the rest of the content will be invisible.
scroll	The overflowing content is clipped, just like hidden, but provides a scrolling mechanism to access the overflowed content.
auto	If content overflows the element's box it will automatically provides the scrollbars to see the rest of the content, otherwise scrollbar will not appear.

HTML & CSS Code

Copy Code

```html
<!DOCTYPE html>
<html>
<head>
  <style>
    .box {
      width: 200px;
      height: 100px;
      border: 2px solid #333;
      overflow: visible;
      background-color: #f97316;
    }
  </style>
</head>
<body>
  <div class="box">This is some long content that
    is going to overflow the container. It will
    go outside the box because the overflow
    value is set to visible.</div>
</body>
```

overflow: visible;

The content is not clipped and will be visible outside the container.

Live Preview

Light

Full Screen

Overflow: visible

In the visiblle value, the overflowing content is not clipped and is visible outside the container.

This is some long content that is going to overflow the container. It will go outside the box because the overflow value is set to visible.

Other overflow Values



visible

Default. Content is visible outside the container.



hidden

Content is clipped and not visible.



scroll

Scrollbars are added to see the content.



auto

Scrollbars appear only if needed.

CSS Display Property

- The CSS specification defines the default display value for all the elements, e.g. the element is rendered as block, while the element is displayed inline.
- Overriding the default display value of an element is an important implication of the display property.
- For example, changing an inline-level element to be displayed as block-level element or changing the block-level element to be displayed as an inline-level element.

Display Block

- The block value of the display property forces an element to behave like block-level element, like a or <p> element.
- The style rules in the following example displays the and <a> elements as block-level elements:
- Example:

```
span {
    display: block;
}
a {
    display: block;
}
```

Display Inline

- The inline value of the display property causes an element to behave as though it were an inline-level element, like a or an <a> element.
- The style rules in the following example displays the <p> and <li> elements as inline-level elements:
- Example:

```
p {
    display: inline;
}
ul li {
    display: inline;
}
```

Display Inline-Block

- The inline-block value of the display property causes an element to generate a block box that will be flowed with surrounding content i.e. in the same line as adjacent content.
- The following style rules displays the and elements as inline-block:
- Example:

```
p {
    display: inline;
}
ul li {
    display: inline;
}
```

Display None

- The value none simply causes an element to generate no boxes at all. Child elements do not generate any boxes either, even if their display property is set to something other than none.
- The document is rendered as though the element did not exist in the document tree.
- Example:

```
h1 {
    display: none;
}
p {
    display: none;
}
```

CSS Positioning

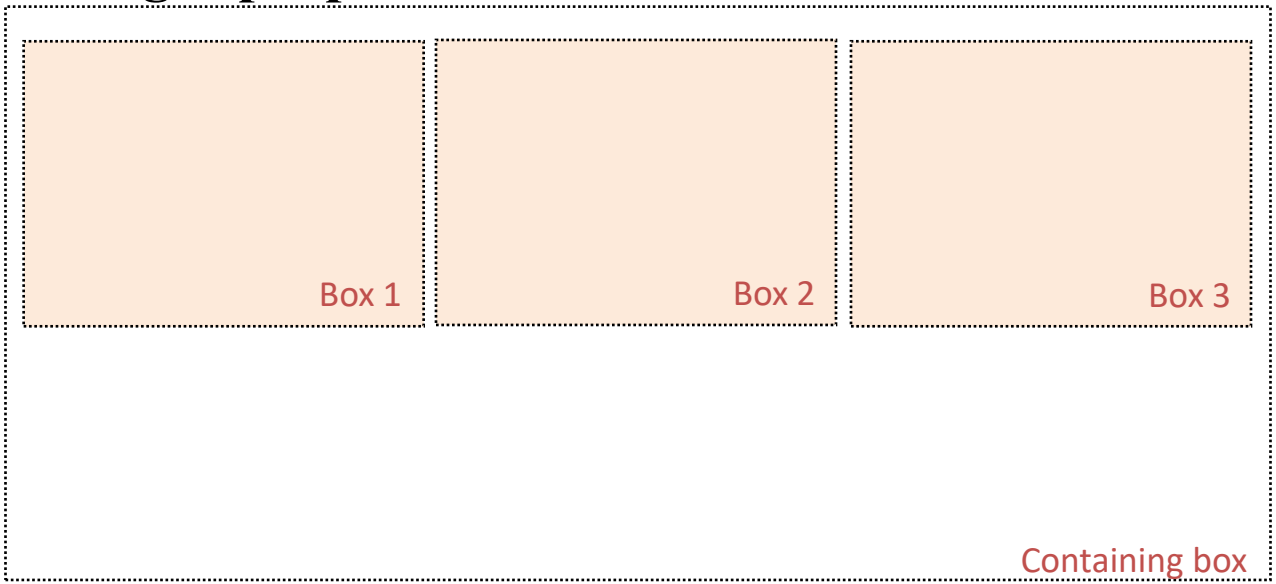
- CSS positioning is about controlling the placement of elements within a web page.
- The position property specifies the positioning type for an element.
- With CSS positioning, you can override the normal document flow.

CSS position Property

- This property can have one of the following values:
  - **static:** This is default. Element is positioned according to the normal document flow
  - **relative:** Element is positioned relative to its normal position in the document flow
  - **fixed:** Element is positioned relative to the viewport
  - **absolute:** Element is positioned relative to the nearest positioned ancestor
- Elements are then positioned to their final location with the **top, bottom, left,** and **right** properties.

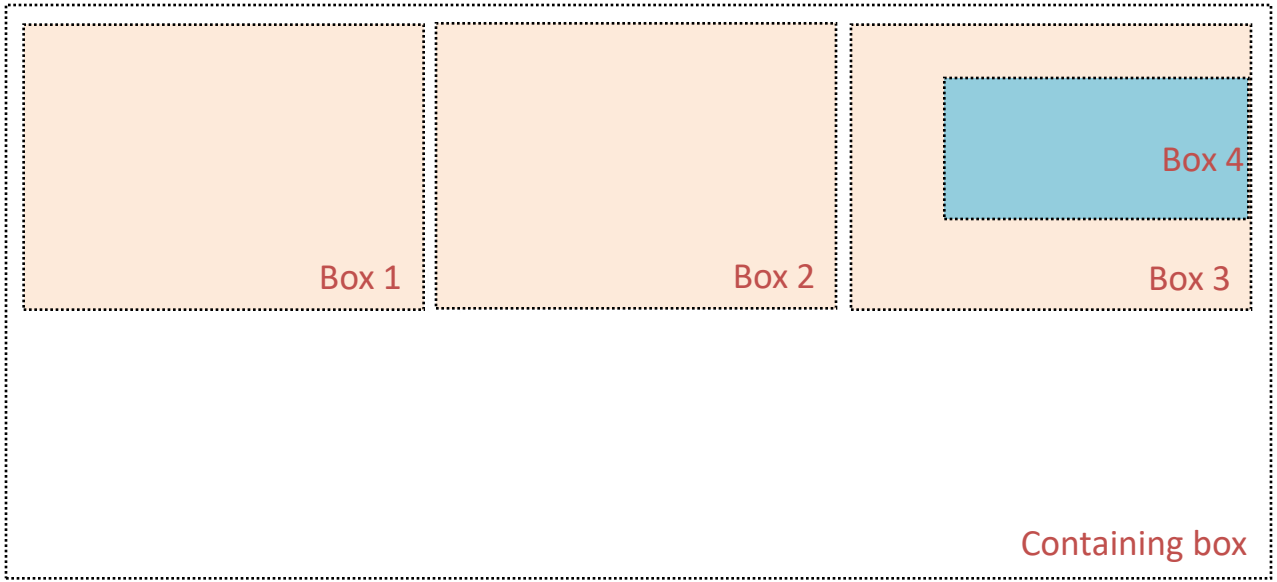
CSS Positioning: Static Positioning

- HTML elements are positioned static by default. A static positioned element is always positioned according to the normal flow of the page.
- Static positioned elements are **not affected by the top, bottom, left, and right properties.**



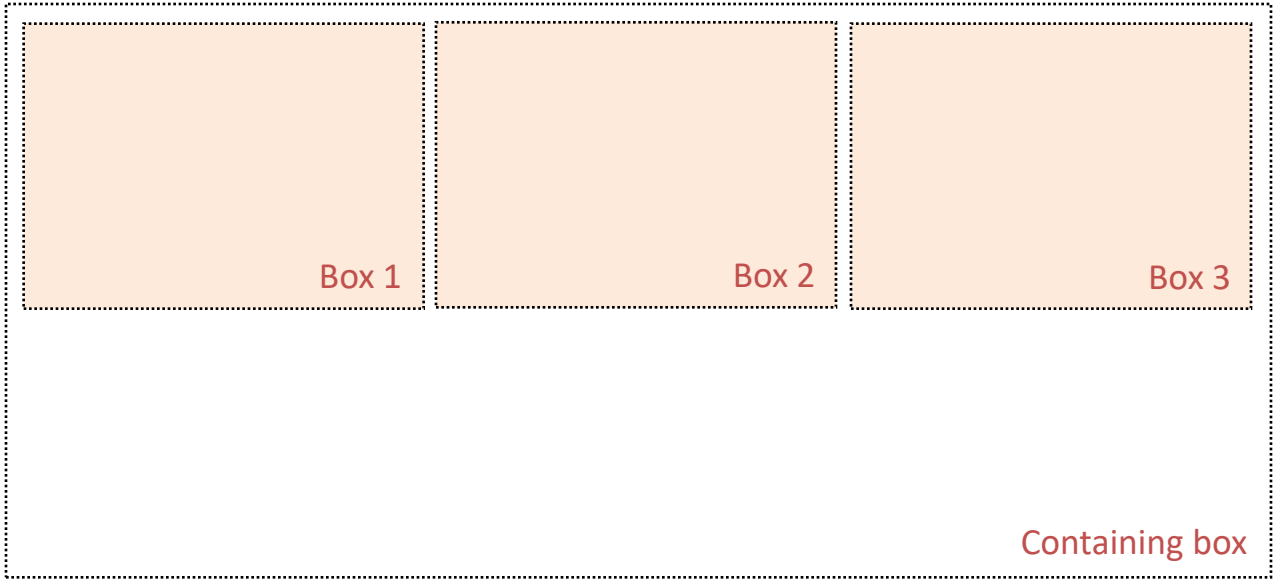
CSS Positioning: Fixed Positioning

- An element with fixed position is positioned relative to the browser window.
- It will not move even if the window is scrolled.



CSS Positioning: Relative Positioning

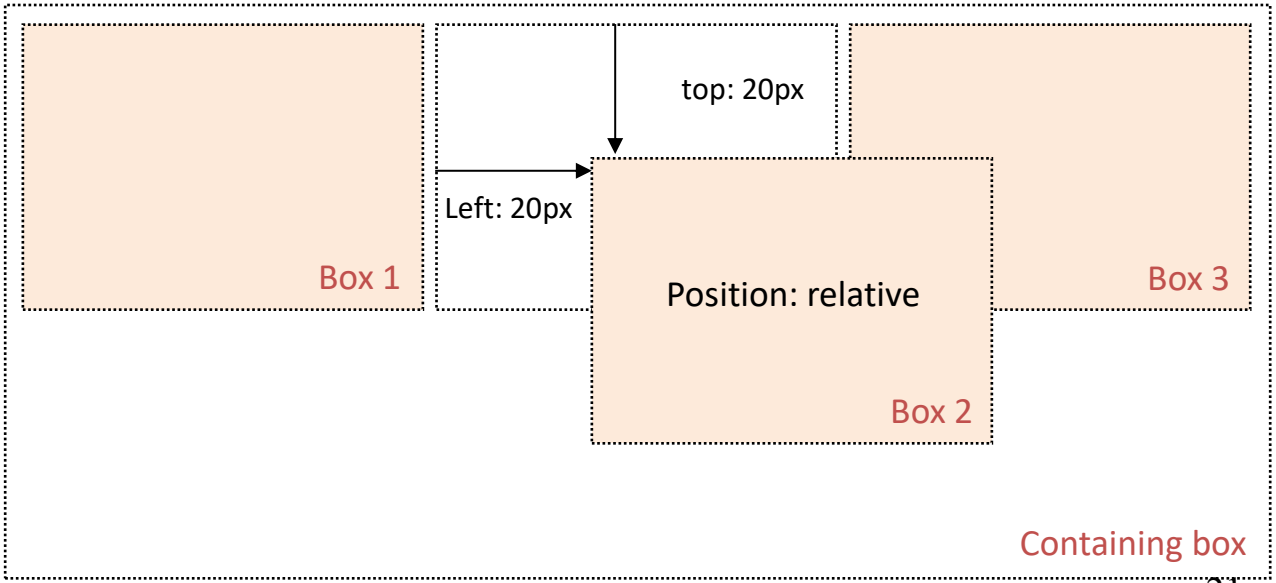
- A relatively positioned element will stay exactly where it is, in relation to the normal flow.
- You can then offset its position “relative” to its starting point in the normal flow:



CSS Positioning: Relative Positioning

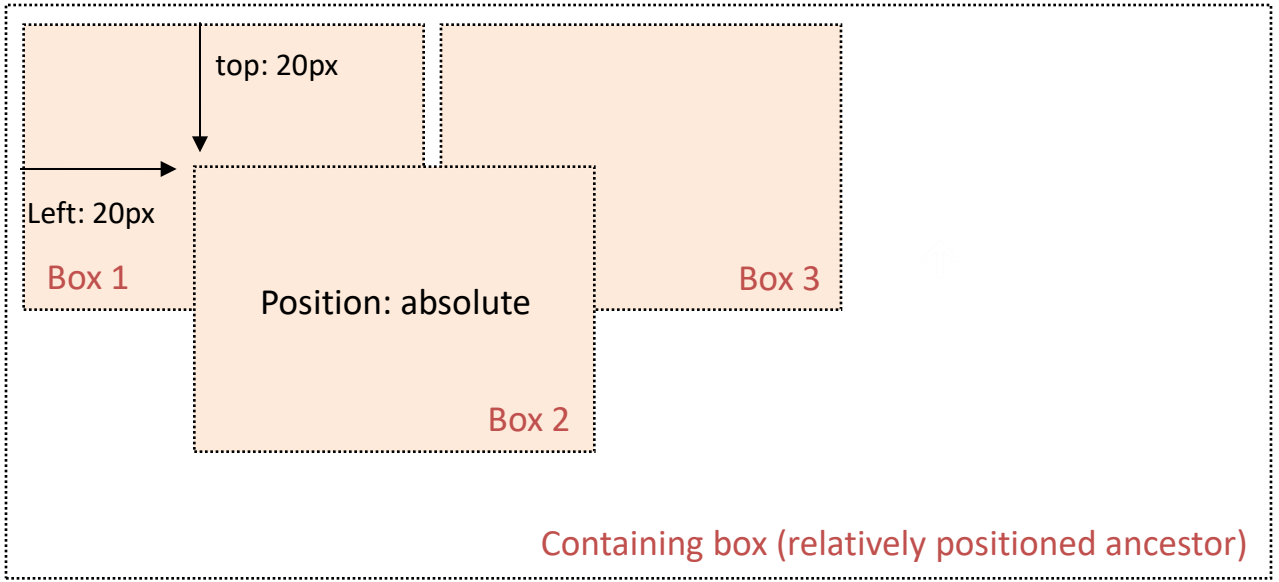
- In this example, box 2 is offset 20px, top and left. The result is the box is offset 20px from its original position in the normal flow. Box 2 may overlap other boxes in the flow, but other boxes still recognize its original position in the flow.

```
#myBox {
    position: relative;
    left: 20px;
    top: 20px;
}
```



CSS Positioning: Absolute Positioning

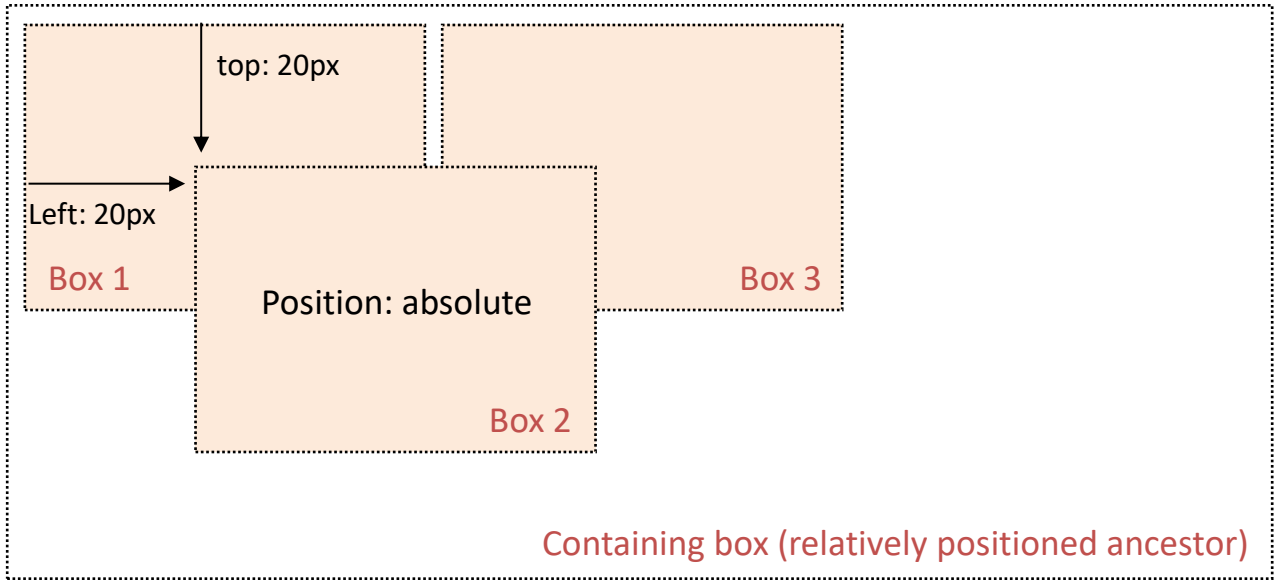
- In this example, box 2 is offset 20px, top and left. The result is the box is offset 20px from its original position in the normal flow. Box 2 may overlap other boxes in the flow, but other boxes still recognize its original position in the flow.



CSS Positioning: Absolute Positioning

- An absolutely positioned box can be offset from its initial position inside the containing block, but other boxes within the block (and still within the normal flow) act as if the box wasn't there.

```
#myBox {
    position: absolute;
    left: 20px;
    top: 20px;
}
```



CSS z-index Property

- The z-index property specifies the stack order of positioned elements.
- The stack order defines which element should be placed in front or behind other elements.
- When elements are positioned, they can overlap other elements.
- An element can have a positive or negative stack order (z-index):

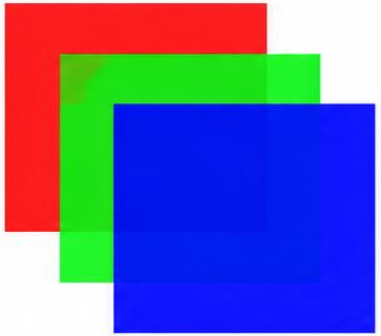
HTML & CSS Code

Copy Code

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="utf-8">
   <title>Example of CSS Layers Effect</title>
   <style>
    .container{
        position: relative;
     }
     .box{
         width: 150px;
         height: 150px;
         opacity: 0.9;
         position: absolute;
     }
     .red{
         background: #ff0000;
         z-index: 1;
     }
     .green{
         top: 30px;
         left: 30px;
         background: #00ff00;
         z-index: 2;
     }
     .blue{
         top: 60px;
         left: 60px;
         background: #0000ff;
         z-index: 3;
     }
   </style>
</head>
<body>
   <div class="container">
      <div class="box red"></div>
      <div class="box green"></div>
      <div class="box blue"></div>
   </div>
</body>
</html>
```

Live Preview

Light Fullscreen



How it Works

All boxes are positioned absolutely inside a relatively positioned container.

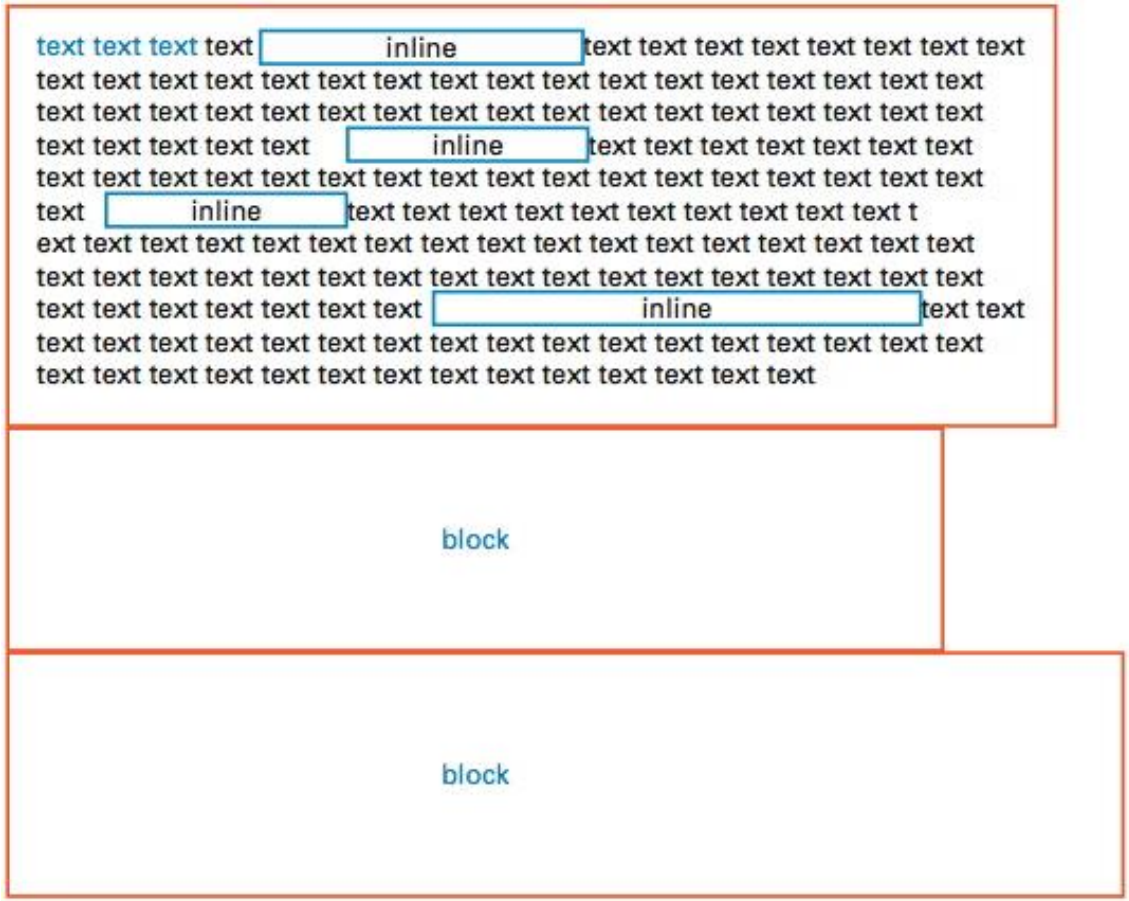
The `z-index` property controls the stack order (which box appears on top).

Stacking Order (z-index)



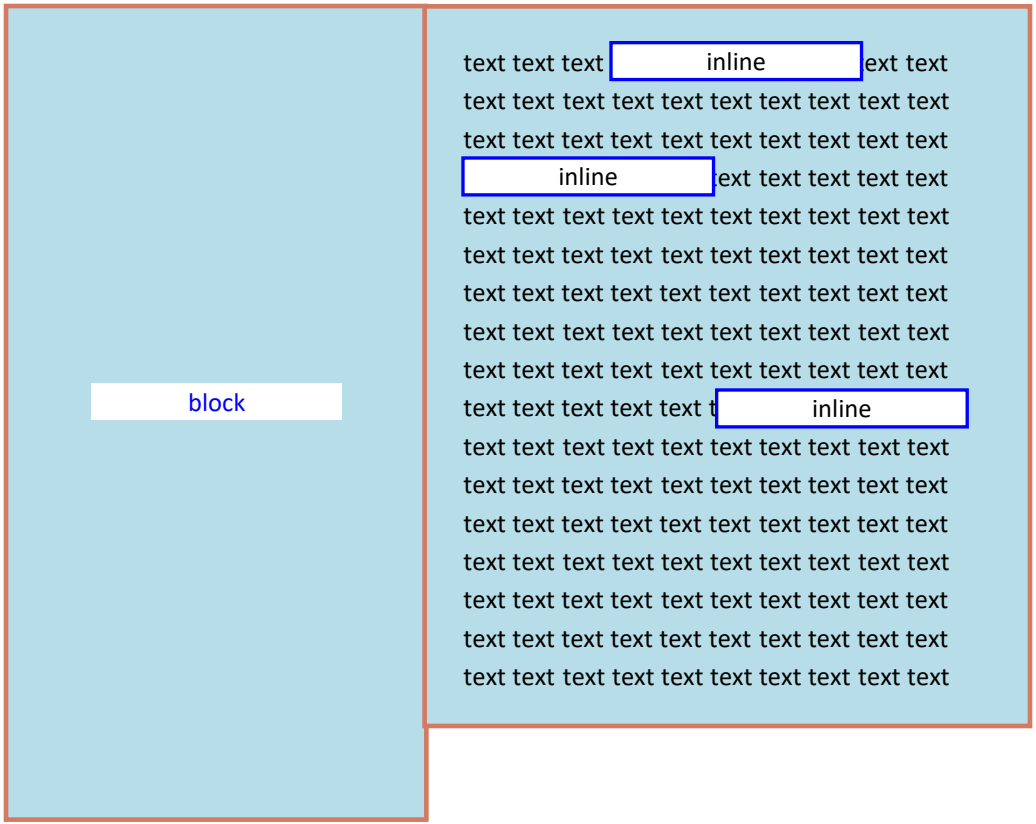
CSS Floats: “Normal Flow”

- CSS boxes for block elements are stacked, one above the other, so that they’re read from top to bottom.
- In CSS, this is said to be the “normal flow”.
- **Note that** CSS boxes for inline elements are placed next to each other, within boxes for block elements, and will normally wrap according to the width of the containing block.


text text text text inline text text text text text text text text
text text text text text text text text text text text text text text text text
text text text text text text text text text text text text text text text text
text text text text text inline text text text text text text text
text text text text text text text text text text text text text text text text
text inline text text text text text text text text text text t
ext text text text text text text text text text text text text text text text
text text text text text text text text text text text text text text text text
text text text text text text text inline text text
text text text text text text text text text text text text text text text text
text text text text text text text text text text text text text
block
block

Floats: Positioning CSS Boxes

- We can position block element boxes side-by-side in CSS using floats.
- Setting the float property to left or right causes a box to be taken out of its position in the normal flow and moved as far left or right as possible.

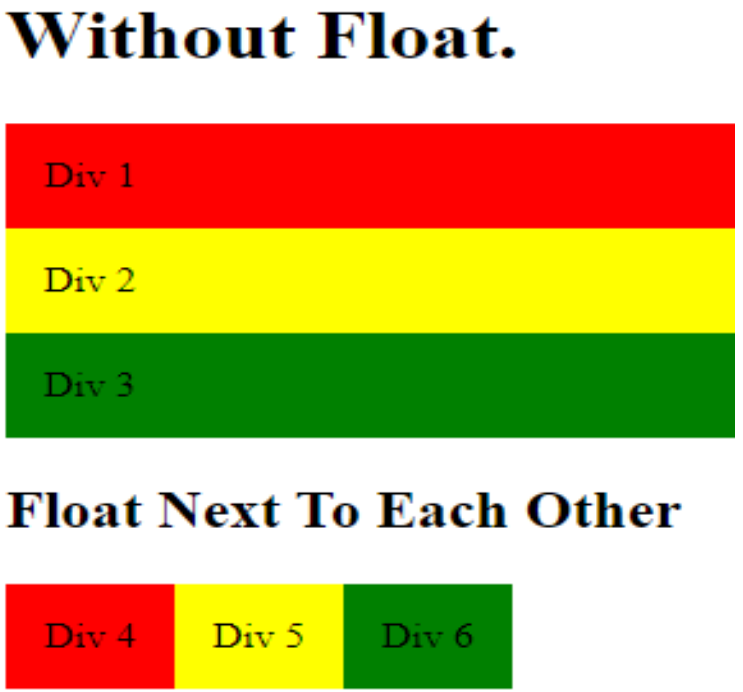

block
text text text inline ext text
text text text text text text text text text text
text text text text text text text text text text
inline ext text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text t inline
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text text

Float Values

- The Float property has three value options:
  - float: left;
  - float: right;
  - float: none;

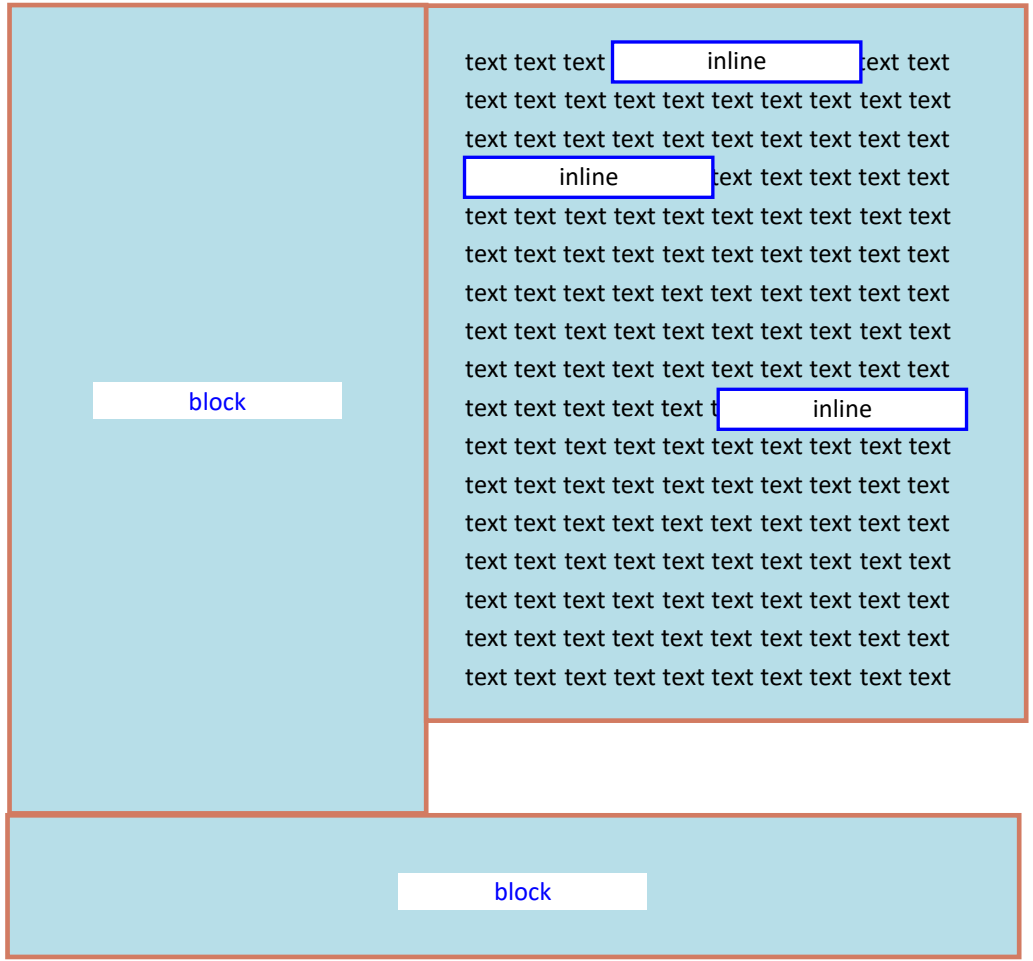
Float Values

- The Float property has three value options:
  - float: left;
  - float: right;
  - float: none;



Restoring the Normal Flow: “Clear”

- To restore the “normal flow”, we can use the clear property.
- The clear property has three value options:
  - clear: left;
  - clear: right;
  - clear: both;
- These specify which side of the element’s box may not have a float next to it.


block
text text text inline ext text
text text text text text text text text text
text text text text text text text text text text
inline ext text text text text
text text text text text text text text text
text text text text text text text text text
text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text
text text text text text inline
text text text text text text text text text
text text text text text text text text text
text text text text text text text text text text
text text text text text text text text text
text text text text text text text text text
text text text text text text text text text text
block

```
<style>
.div1 {
  float: left;
  padding: 10px;
  border: 3px solid #73AD21;
}

.div2 {
  padding: 10px;
  border: 3px solid red;
}

.div3 {
  float: left;
  padding: 10px;
  border: 3px solid #73AD21;
}

.div4 {
  padding: 10px;
  border: 3px solid red;
  clear: left;
}
</style>
```

Without clear

div1 div2 - Notice that div2 is after div1 in the HTML code. However, since div1 floats to the left, the text in div2 flows around div1.

With clear

div3

div4 - Here, clear: left; moves div4 down below the floating div3. The value "left" clears elements floated to the left. You can also clear "right" and "both".

Box Sizing

- **Redefining Box Width with Box-Sizing:** By default, the actual width or height of an element's box visible/rendered on a web page depends on its width or height, padding and border property.
- For example, if you apply some padding and border on a element with 100% width the horizontal scrollbar will appear, as you can see in the example below.

```
.box {
    width: 100%;
    padding: 20px;
    border: 5px solid #f08080;
}
```

Box Sizing

- CSS3 introduces the box-sizing property to solve this problem and make the CSS layouts much more simple and intuitive.

- The box-sizing property alter the default CSS box model in such a way that, any padding or border specified on the element is laid out and drawn inside of the content area, so that the rendered width and height of the element is equal to the specified CSS width and height properties.

Box Sizing

- Example:

```
.box {
    width: 100%;
    padding: 20px;
    border: 5px solid #f08080;
    box-sizing: border-box;
}
```

Box Sizing

- CSS3 introduces the box-sizing property to solve this problem and make the CSS layouts much more simple and intuitive.

- The box-sizing property alter the default CSS box model in such a way that, any padding or border specified on the element is laid out and drawn inside of the content area, so that the rendered width and height of the element is equal to the specified CSS width and height properties.

More CSS Important Topics

- **CSS Flexbox**
- **CSS Grid**

Thank You!